Lesson 9 — What Each Notebook Shows

Notebook guide, lesson 9 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

27 November 2026 · three notebooks

Contents

Notebook 01 — `01_from_one_line_to_a_hidden_layer` **1**
From one line to a hidden layer 1

Notebook 02 — `02_a_network_in_keras` **2**
A network in Keras, on handwritten digits 2

Notebook 03 — `03_training_in_practice` **3**
Training in practice 3

A short guide to this lesson's notebooks. For each one: what it is for, the concepts in the order they appear, the numbers worth noticing, and **the two-minute version**, a summary to read before running it or to come back to afterwards. Every number here comes from the committed output of the notebook it belongs to.

Notebook 01 — `01_from_one_line_to_a_hidden_layer`

From one line to a hidden layer

What it is for. To see what one unit can do, what it cannot, and what changes when several units sit side by side in a hidden layer. scikit-learn throughout; no TensorFlow needed.

The data. Meridian Instruments' acceptance test, 3,000 sensors on two axes, split 2,250 to train and 750 to test; and the two-channel drift problem, 800 sensors in four clouds.

The concepts, in order.

1. **Look first.** Standardised, the accept region is a circle of radius 1.25. The test rig records 3% of verdicts wrongly, so 0.97 is the ceiling.
2. **One line.** Logistic regression scores **0.5507** on the test sensors, against **0.5480** for always saying "accepted". The fit did not fail: a line cannot enclose a region.
3. **One unit by hand.** With $w = (1, 2)$, $b = -1$: the point $(1, 1)$ gives $\hat{y} = 0.8808$, the point $(1, 0)$ sits on the line at exactly 0.5.

4. **The drift problem.** A line scores exactly **0.5000**. Two ReLU units written down by hand score **0.9938** on all 800 sensors, and the hidden-space plot shows why: the hidden layer moved the data until one line was enough.
5. **Training finds the weights, sometimes.** `MLPClassifier`, twenty runs per size: two hidden units reach 0.99 in **6 of 20** runs, four in **17**, eight in **all 20**.
6. **Back to the acceptance test.** Mean of five runs: 1 unit 0.6048, 2 units 0.7392, 4 units 0.8821, **8 units 0.9445**, 16 units 0.9549. The boundary grows from a line to a strip, a four-sided fence and an eight-sided one.
7. **What it was worth:** from 0.5507 to 0.9445, **39.4 points**.

The two-minute version. *One unit is one straight line, and on a problem whose answer is a disc a line scores what saying "accepted" to everyone scores. A hidden layer moves the data into new coordinates where a line is enough; two units written by hand solve the drift problem. Training finds such weights more reliably as the layer widens, and eight hidden units take the acceptance test from 0.55 to 0.94, 39 points, three short of the ceiling.*

Notebook 02 — `02_a_network_in_keras`

A network in Keras, on handwritten digits

What it is for. To build, train and read a network layer by layer in Keras, the library lesson 10 uses, and to measure what a hidden layer is worth on real data.

The data. The 8×8 handwritten digits bundled with scikit-learn: 1,077 to train, 360 to validate, 360 to test.

The concepts, in order.

1. **The network in three lines**: 64 inputs, a hidden layer of 32 ReLU units, 10 softmax outputs; **2,410** parameters, checked by hand.
2. **Training, watched**: 34 mini-batches of 32 per epoch, 2,040 steps in 60 epochs. Validation accuracy 0.2667 after one epoch, **0.9639** after sixty (one run).
3. **Softmax**: ten probabilities adding up to 1. An easy 6 gets 0.991; the mistake the network was least sure about gives no digit more than a third; and a 4 is called a 7 with probability **0.920**: confidence is not correctness.
4. **Is the hidden layer worth it?** Mean of three runs on the validation set: softmax alone **0.9324**, one hidden layer of 32 **0.9602**, of 64 0.9611.
5. **The number of the lesson**: on the sensors the hidden layer is worth **39.4 points**, on the digits **2.8**.

The two-minute version. *Keras lists the layers, compile picks the loss and the optimiser, fit runs the training loop in mini-batches. Softmax turns ten scores into ten probabilities, which say how sure the network is, not how right. On the digits a hidden layer adds 2.8 points over softmax alone, against 39 on the sensors: pixels push the answer more or less independently, the sensor's two offsets only matter together. Fit the linear model first; the gap tells you whether you needed a network.*

Notebook 03 — `03_training_in_practice`

Training in practice

What it is for. The four things that most often go wrong, each shown on the digits, and the property of dense networks lesson 10 starts from.

The concepts, in order.

1. **The learning rate**, two hidden layers of 64, plain SGD, 60 epochs, one run each: $= 0.001$ reaches **0.4806** and is still learning; $= 0.5$ reaches **0.9722**; $= 2.0$ stays at **0.1000**, guessing.
2. **Memorising**: two layers of 512 on 300 digits, **301,066** weights and biases. Training accuracy 1.000; validation loss lowest at **epoch 13**, then 1.4 times higher by epoch 200, while validation accuracy (0.9472) hides it.
3. **Two remedies**, three runs each, on the test set: no remedy 0.9426, early stopping 0.9463 (39 epochs), early stopping and dropout 0.9481. The differences sit inside the spread; early stopping certainly saved time.
4. **Six sigmoid layers**: validation accuracy **0.1000** after 80 epochs; the same with ReLU **0.9611**. The sigmoid's slope is at most 0.25, and the blame shrinks at every layer: the vanishing gradient.
5. **Shuffled pixels**: 0.9602 as written, 0.9630 shuffled, three runs each. A dense network does not notice.

The two-minute version. *Try learning rates a factor of ten apart before blaming the network. A network with far more weights than examples memorises, and the validation loss says so before the accuracy does; early stopping and dropout help a little and early stopping saves time. Deep sigmoid stacks never learn because the blame shrinks at every layer; ReLU fixes it. And a dense network cannot tell a digit from its pixels shuffled, which is where lesson 10 begins.*